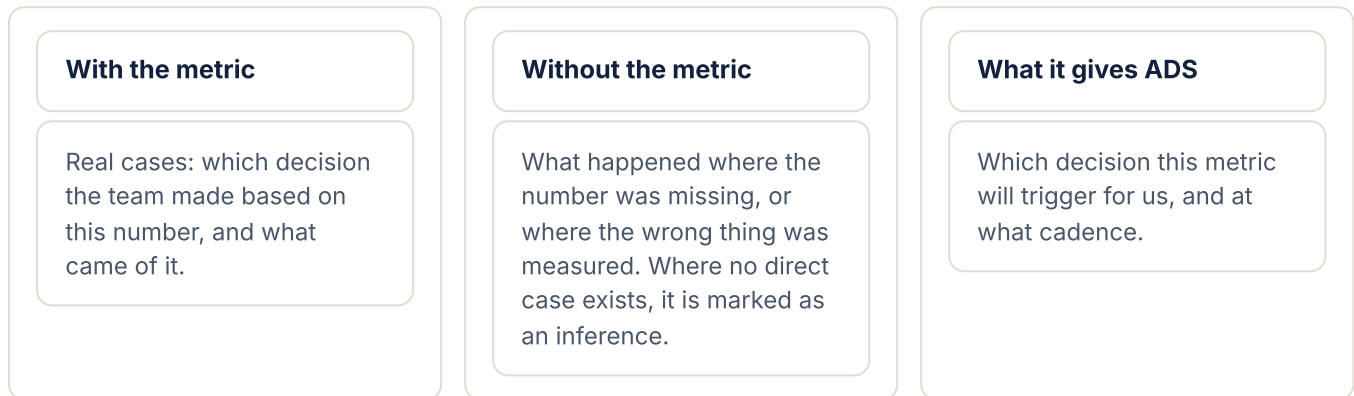


What each metric gives you

The fifteen metrics of the ADS working set, and for each one: what happened at teams that measured it, and what happens at teams that do not. All cases come from published company material and industry reports; sources are listed at the end.



#	Metric	Question	Key case
1	Adoption rate	Adopt	Pinterest: team spread from 2% to 50% → targeted training
2	Accessibility pass rate	Quality	Sparkbox / Carbon: the system speeds you up, accessibility does not improve by itself
3	Breaking change rate	Governance	Nathan Curtis: the team writes its own definition of a breaking change
4	Estimated hours saved	Value	IBM: 2,010 hours saved across 18 interfaces
5	Detach & override rate	Fidelity	Google: a button with high usage and high detach
6	Figma-code parity	Fidelity	Pinterest: the gap between Figma and code revealed a component limitation
7	Token adoption in code	Fidelity	Shopify: new styles updated 86% of the admin automatically
8	Consumer satisfaction	Satisfaction	Lucid: the survey died after two quarters
9	Lead time for requests	Service	Knapsack: cycle time as the most honest proxy for value
10	Bug resolution time	Service	Headway: "noise is health, silence is the danger"
11	Bugs per component	Quality	Optimizely: data picked Icon and Button for migration first

12	Component readiness	Quality	Sprout Social: Healthy / Withering / Dormant statuses
13	Version adoption lag	Governance	Spotify: daily snapshot of every team's library version
14	Blast Radius	Governance	Curtis: the designer "sprays" upgrades without knowing their cost
15	Library utilization	Adopt	Booking.com: 146 components before consolidation

Adoption rate

Engineering · monthly · KPI

What it shows. The share of UI built from ADS: in code, usages of ADS components in Angular templates over all UI components; in Figma, ADS instances over all instances in product files. Per product and overall.

WITH THE METRIC

Pinterest, Gestalt. A nightly script counted the share of layers from the system per team. The spread turned out to be ~2% to ~50%. The team stopped training everyone and went to specific teams instead. A side effect: PageHeader was inserted often in Figma but barely used in code — a component limitation, not team reluctance. [Figma blog](#), 2023

Uber, Base. Automated count of "Base vs looks-like-Base" per screen, Jira tickets, a dashboard per app. Over 2023–2024 adoption in Rider grew by 17 pp; 100+ screens migrated. [Uber Engineering](#), 2024

IBM, Carbon for IBM.com. Share of traffic on Carbon-built pages: 6.2% → 44.8% during 2021. The metric was chosen because it "resonates with executives". [Knapsack](#), 2022

WITHOUT THE METRIC

Netflix, Hawkins. "We haven't put an emphasis on tracking the system, but are starting to think about metrics now." And right after: "Senior leadership might want to see adoption metrics as a way of understanding the ROI." The question comes from the top; there is no answer. [The Design System Guide](#), interview

Robin Cannon, Carbon co-lead. Until adoption was tied to benefits, leadership called it "a vanity metric without context". [robin-cannon.com](#), 2022

Dan Mall. Has seen teams with 90–100% adoption and the same inefficiencies as before the system. Adoption can be mandated; value cannot. [danmall.com](#)

WHAT IT GIVES ADS

A product with a low share gets an adoption plan; recurring custom components go into the ADS backlog. This is the one metric leadership will ask for unprompted, so it goes first in the report — but never alone.

Accessibility pass rate

Engineering · every release · KPI

What it shows. The share of components that pass axe on their Design Portal page in all variants plus a manual keyboard and screen-reader check.

WITH THE METRIC

Sparkbox, Carbon experiment. Eight developers built a form twice: 47% faster with the system (median 4.2 h → 2 h), visual consistency improved for 5 of 8, accessibility — "mixed results". The system speeds you up, but a11y does not improve on its own; it has to be checked separately. *Sparkbox Foundry*

Uber, Base. The adoption dashboard carries a11y issue severity per screen; violations automatically become Jira tickets. *Uber Engineering, 2024*

Industry. Accessibility compliance is the fourth most common design system metric: 36% of teams in 2026. *zeroheight DS Report 2026*

WITHOUT THE METRIC

Sparkbox, maturity model. At maturity stage three the main measurement focus is "accessibility regressions". Without a component-level check, a regression ships to every product that uses the component at once. *Sparkbox Maturity Model*

Inference. For telehealth, WCAG AA is a requirement. A component checked once at creation and never again guarantees nothing after a few releases. Public failure cases are rare because nobody writes about them.

WHAT IT GIVES ADS

A component that fails does not get Stable status; violations go into the next sprint. For leadership this answers the compliance question that legal would otherwise ask.

Breaking change rate

Engineering · every release · KPI

What it shows. The number of breaking changes per release and the share of major releases per quarter. A breaking change is an API change or a noticeable visual change of a component.

WITH THE METRIC

Nathan Curtis, EightShapes. "There is no single, global definition of a breaking change — you must write this definition yourself and socialize it within your organization."

Breaking-change ideas go into a separate bucket and ship as one deliberate major release, not one by one. *Versioning Design Systems*

Spotify, Encore. Daily snapshot of every team's library version shows not only upgrades but downgrades — a direct signal that a release broke something. *Spotify Design*

Industry. At well-adopted systems 90% of respondents report high stability, 29 pp more than the rest. Stability and trust go together. *zeroheight DS Report 2025*

WITHOUT THE METRIC

Nathan Curtis. "The designer sprays style and component upgrades across a new feature, completely unaware of the upgrade costs it triggers across our whole app." Without counting breaking changes, product teams pay their cost and the system team does not see it. *Versioning Design Systems*

Inference. The consequence shows up in metric 13: teams stop upgrading because every upgrade is a risk. Fixes and a11y patches never reach them.

WHAT IT GIVES ADS

A breaking change is allowed only with a migration guide and a codemod; a rising rate is a signal to revisit API design. The team has already agreed the definition; it remains to write it into the contribution guide.

Estimated hours saved

What it shows. An estimate of hours saved: component usages × average estimate of building an equivalent from scratch, minus ADS team hours. The number is model-based and always comes with its assumptions.

WITH THE METRIC

IBM, pattern efficiency formula. (Design + development + maintenance) × number of interfaces without the system versus the same with the system plus import time. Across 18 interfaces: 2,484 h vs 474 h — 2,010 hours saved, 81%. Knapsack, Design System Insights: ROI

Figma, controlled experiment. Designers built screens with and without a system: 34% faster with it. The authors call this a ceiling because the system fit the task perfectly. Figma blog, 2019

Sparkbox, Carbon. 47% faster on a real task, median 4.2 h → 2 h. Sparkbox Foundry

WITHOUT THE METRIC

Knapsack. "To get buy-in, stakeholders have to be willing to take a leap of faith." At the expansion stage "you might not be able to show great returns on hours saved, but you will be able to prove adoption — bring testimonials." Without a model, the budget conversation runs on faith. Design System Insights: ROI

Industry. ROI is measured by 5% of teams. It is the rarest metric and the most frequent leadership question. zeroheight DS Report 2026

WHAT IT GIVES ADS

Justifies the team budget once a quarter. Not used to manage sprints. The key assumption is the average build-from-scratch estimate; we take it from a one-off benchmark or engineers' estimates and always show it next to the number.

Vendor figures such as "15% faster time-to-market" or "\$3M saved" without methodology are deliberately left out.

Detach & override rate

Design + Engineering · monthly

What it shows. In Figma, detaches ÷ inserts per component per month; in code, the share of ADS component usages with style overrides (::ng-deep, !important, custom classes on top).

WITH THE METRIC

Google, Material. Analytics showed the button had both high usage and high detachment. That was read as a signal that the master component was too complex, and it was simplified. [Figma blog, 2019](#)

athenahealth, Forge. "If someone in our organization is detaching a component, I want to know why." Monthly review of a dashboard with inserts and detaches. [Figma blog, 2025](#)

Figma, guidance. A detachment means one of three things: a bug, an enhancement request, or an unexpected combination of elements. A high detach rate is not always negative — it is the earliest signal of a need. [Design Systems 104](#)

Microsoft, Fluent. "We see which components aren't used and which are often detached and may need updating." [Figma blog, 2019](#)

WITHOUT THE METRIC

"97% connected". The system was formally installed almost everywhere, while a Slack channel next to it traded ways to work around it. Adoption green, product not. Detach and override are the only way to see this in data. [dnsk.work](#)

Mews. The team's first metric — "number of deviations" — did not survive: stakeholders did not understand it because it measured debt, not value. A lesson in framing: detach is an operational team metric, not a leadership KPI. [developers.mews.com, 2025](#)

WHAT IT GIVES ADS

A component with a high rate goes into root-cause research and API or variant rework. This is the main leading signal that a component does not cover a real need, and the main source of backlog items.

Figma-code parity

Design + Engineering · monthly

What it shows. The share of components whose Figma component is linked to code via Code Connect and whose set of properties matches the inputs in code, names included.

WITH THE METRIC

Pinterest, Gestalt. Comparing Figma usage and code usage of one component (PageHeader) exposed a gap: designers wanted it, engineers could not apply it. The gap between the two sides is the parity signal. [Figma blog, 2023](#)

Uber, Base. Claims "4X fewer visual parity issues" after moving to a system with a single source of components. [Uber Engineering, 2024](#)

Netguru, audit practice. Design-code parity is part of the baseline design system audit alongside usage and a11y. [netguru.com](#)

WITHOUT THE METRIC

Inference. There are almost no public failure stories here, which is telling in itself: Figma-code drift is rarely measured; it is discovered at handoff, when a designer used a variant that does not exist in code. Each such discovery is rework or custom code that later shows up as a detach.

Tool limitation. No tool verifies structural parity of variants and props automatically; Code Connect coverage is the pragmatic proxy. [Tooling review, 2026](#)

WHAT IT GIVES ADS

A mismatch becomes an alignment ticket before the next release; new components are not published without parity. For a team that owns both Figma and code, this is a metric of its own discipline.

Token adoption in code

Engineering · every release

What it shows. The share of style values in ADS and product repositories taken from tokens rather than hardcoded. Checked by a stylelint rule.

WITH THE METRIC

Shopify, Polaris. When the admin was 86.6% covered by the system and its styles, shipping the updated styles changed "about 86% of the admin automatically". Tokens turned a redesign from a project into a release. [Shopify UX, Medium](#)

Productboard. ESLint and stylelint rules flag font-size and line-height not taken from tokens; the warning count is the denominator of the "custom typography" metric used to track migration progress. [Productboard blog, 2021](#)

Pinterest. On top of adoption, they lint text-style usage and the prevalence of raw hex codes. [Figma blog, 2023](#)

WITHOUT THE METRIC

Google, Material, before the system. An audit found 3,000 versions of one button. That is what a codebase looks like when colours and spacing live separately in every file. [designsystems.com](#)

Inference. Without a token share, any brand or theme change is a manual sweep of every repository. That is the difference between "86% updated automatically" and a quarter of work.

WHAT IT GIVES ADS

Violations in ADS block the merge; the trend per product shows where help with token migration is needed. The ADS repository already has semantic tokens and themes — the metric checks that they are actually used.

Consumer satisfaction

What it shows. Designer and engineer satisfaction with components, documentation and response speed: a 5–7 question survey with one open question, reported separately for the two segments.

WITH THE METRIC

Pinterest, Gestalt. A sentiment survey twice a year with the research team since 2020; engineer sentiment 95.7%, designer sentiment 95.2%. [Knapsack, interview](#)

Shopify, DX survey. Twice a year, each half of developers answers once a year to avoid fatigue; stable questions for year-over-year trends; "if developers don't think it improved, you haven't solved it". [getdx.com](#)

Atlassian. Triangulates opt-in/opt-out of changes, product NPS changes after updates and qualitative sessions. [UXPin, interview](#)

WITHOUT THE METRIC

Microsoft, Fluent. "There are so many teams using the Fluent Design System, and we don't always get feedback from them about what's working." Analytics replaces the missing conversation, but does not say why. [Figma blog, 2025](#)

Lucid — measured the wrong thing. Tried to measure "engagement" via Slack comments and a three-question quarterly survey. Response rate fell after two quarters. The team's conclusion: "Measuring the wrong thing and holding people accountable for it can introduce incentives that undermine your whole goal." [Lucid Tech Blog, 2023](#)

Omlet. If you rely on feedback alone, you hear the "pro users" — the people who already contribute. [omlet.dev](#)

WHAT IT GIVES ADS

The three main pains from open answers become next quarter's goals. The survey is short and infrequent, and the wording does not change — otherwise Lucid's story repeats.

Lead time for component requests

What it shows. The median time from ticket creation to release in both npm and Figma. Slices: new component, change, feedback.

WITH THE METRIC

Knapsack. "We often look at production cycle time because it is easy to measure and directly tied to the value." The system team's response speed is what product teams feel every day. [knapsack.cloud](#)

GitHub, Primer. Public commitments: weekly backlog triage, PR response "within a day or two", minor releases weekly, majors twice a year. The commitment is measurable because the times are named. [Primer CONTRIBUTING.md](#)

Nathan Curtis, OKRs. KR "Predictability: the system publishes every minor release on a quarterly basis within 48 hours of announced date and time." [Measuring Design System Success](#)

WITHOUT THE METRIC

Pinterest — what happens while a team waits. Local components in product files are what teams built themselves instead of waiting for the system; Pinterest looks for them specifically to absorb them. Each one is a lost request. [Figma blog, 2023](#)

Inference. Without a lead-time median, the argument "you are slow" versus "we are overloaded" has no arbiter. With it, the conversation is about WIP limits and priorities, not impressions.

WHAT IT GIVES ADS

A rising median triggers a review of WIP limits and prioritization; the value is published to consumers as an SLA expectation. This is the main "we are a good service" metric in product teams' eyes.

Bug resolution time

What it shows. The median time from bug report to closure for ADS bugs.

WITH THE METRIC

Uber, Base. Violations found by the automated traversal become Jira tickets immediately — the bug queue is visible and measurable rather than living in Slack. [Uber Engineering, 2024](#)

Headway. "Bug reports count as a positive signal. Noise is health, silence is the danger." A team that receives and quickly closes bugs is a team people trust. [headway.io](#)

Jira, the tool. The Control Chart computes median and spread by issue type with no extra setup. This is the cheapest metric in the set. [Atlassian docs](#)

WITHOUT THE METRIC

Inference from the Spotify case. The version downgrades Spotify sees are teams that rolled back rather than wait for a fix. The longer a bug lives in a shared component, the more products build a workaround around it — and the workaround stays after the fix. [Spotify Design](#)

Inference. Without a median, bug priority is set by the loudest reporter. With a median and Blast Radius, by the number of affected products.

WHAT IT GIVES ADS

Bugs with a high Blast Radius take priority over new features; a rising median signals a lack of support capacity and is an argument for resources.

Bugs per component

Engineering · monthly

What it shows. The number of bugs per component per quarter, normalized by its usage.

WITH THE METRIC

Optimizely, Axiom. A team of 3 developers and a designer used per-component data to pick Icon and Button first for the React 18 migration: they had the most usage and the most problems. omlet.dev, 2023

Sprout Social, Seeds. Every component page carries "known issues" and "planned enhancements" as part of its health passport. chsmc.org, 2020

Romina Kavcic. Among a system's warning signs — "many bug reports" against the same components. *The Design System Guide*

WITHOUT THE METRIC

Inference. Quality investment (tests, visual regression, refactoring) is spread evenly or goes where people complain loudest. Three components usually produce most of the bugs, and without a per-component breakdown this is invisible.

Link to metric 5. The component with a high detach rate and a high bug count is the same component. The two metrics confirm each other.

WHAT IT GIVES ADS

The top 3 components by bugs get a quality investment: tests, refactoring, visual regression. The Component field in Jira is the only setup needed.

Component readiness

Design + Engineering · every release

What it shows. The share of components meeting the Definition of Done: documentation with examples and do/don't, documentation freshness, tests, a11y, Code Connect, empty and overflow content states, responsiveness, localization. Published as a status on the component page.

WITH THE METRIC

Sprout Social, Seeds. Every component page carries its publish date and last change (from GitHub), presence in the Figma kit, a mobile counterpart, known issues. Statuses Healthy / Withering / Dormant. Stale pages stopped being invisible. [chsmc.org](#), 2020

IBM, Carbon. A new component is accepted into the system only with a spec, usage documentation and working code. The Definition of Done is part of the contribution process. [Carbon contributing](#)

Industry. Among teams with KPIs, documentation ranks second after adoption; zeroheight names poor documentation a leading cause of low adoption. [zeroheight](#), [How We Document](#) 2022

WITHOUT THE METRIC

Sprout Social, before the passports. Pages "withered": the component changed in code, the documentation stayed old, and trust in the portal fell. That is exactly what made the team introduce statuses. [chsmc.org](#), 2020

Industry. 91% of teams track nothing about their documentation. [zeroheight](#), [How We Document](#) 2022

WHAT IT GIVES ADS

A component does not get Stable status until the checklist is complete; the share of Stable is a quarterly goal. Eight separate catalog metrics (documentation, tests, i18n, responsiveness, content states) become items of one checklist on the component page in the Design Portal.

Version adoption lag

Engineering · monthly

What it shows. The time from a new major release until 80% of consumers have moved to it; slice: time to zero usages of a deprecated component.

WITH THE METRIC

Spotify, Encore. Daily statistics on which team runs which library version; comparison of adoption, upgrades and downgrades over time. Low-usage components and icons become deprecation candidates. *Spotify Design*

Brevo, Indigo. Version compliance and migration ratio (new components to old ones) computed by a script twice a month.

engineering.brevo.com, 2023

Nathan Curtis, OKRs. "Adoption Age: system version dependencies are no more than 6 months old for flagship products and 12 months old for secondary products."

zeroheight: "adopt major updates within six months." *EightShapes; zeroheight*

Squarespace, Swiggy. Track version migration as a metric in its own right. *Figma blog, 2025*

WITHOUT THE METRIC

Nathan Curtis. Before deprecating, "audit product code to find instances where it's used, and if used broadly, communicate with those teams prior to making a decision." Without version data, deprecation is a letter to nowhere. *Versioning Design Systems*

Inference. The system team supports as many versions as live in products. If nobody knows that number, support cost grows unnoticed and fixes never reach the laggards.

WHAT IT GIVES ADS

A lag is a signal that migration is too expensive: a codemod, a guide or hands-on help is needed. The package.json version snapshot is part of the same script that computes adoption.

Blast Radius

Design + Engineering · weekly

What it shows. The number of Figma files, Figma projects, repositories and unique stakeholders affected by a change to a component; Low, Medium and High levels define the change process.

WITH THE METRIC

Omlet, customer practice. "As components or styles are adopted by more teams, the impact of a change becomes larger and trickier to assess." A dependency tree answers "what breaks if I change this".

omlet.dev

Nathan Curtis. Auditing usage and talking to teams before deprecation is a mandatory step. Blast Radius makes that audit continuous rather than one-off.

[Versioning Design Systems](#)

Uber, Base. Automated traversal plus a gate in the feature readiness process — the metric earned the right to stop changes.

[Uber Engineering, 2024](#)

WITHOUT THE METRIC

Nathan Curtis. "The designer sprays upgrades across a new feature, completely unaware of the upgrade costs it triggers across our whole app." The cost of a change is invisible to the person making it.

[Versioning Design Systems](#)

Mews. Stakeholders did not understand the metric until it was about their product. That is why Blast Radius is counted in stakeholders: "5 product owners affected" is clear to everyone.

developers.mews.com

WHAT IT GIVES ADS

Defines the process: Low — the designer fixes it alone, Medium — alignment with affected teams, High — an RFC and review by all stakeholders. It is also an ROI argument: a component with a high radius is genuinely reused; a core component with a zero radius is a reason to think.

Library utilization

Design · quarterly

What it shows. The share of library components with at least one insert or usage in 90 days; slice: variant usage frequency.

WITH THE METRIC

Spotify, Encore. Low-usage components and icons become deprecation candidates based on data, not gut feeling. [Spotify Design](#)

Booking.com. 146 components before consolidation; libraries used by hundreds of teams. Consolidation is impossible without usage data. [ardakaracizmeli.com](#)

Microsoft, Fluent. "We see which components aren't used." [Figma blog](#), 2019

zeroheight. "A design system with 300 components isn't necessarily better than one with 60." Component count is a vanity metric; the share in use is not. [zeroheight blog](#)

WITHOUT THE METRIC

Google, Material, before the system. 3,000 versions of one button — the extreme case of what a library becomes when nothing is ever removed. [designsystems.com](#)

Inference. Every unused component still costs: it is updated when tokens change, checked for a11y, kept in parity with Figma. Without the share of components in use, that cost is invisible and grows with every release.

WHAT IT GIVES ADS

Components and variants unused for two consecutive quarters are deprecation candidates. This is the only metric in the set that reduces the team's workload rather than adding to it.

Sources

All cases come from the material listed below. Articles on Medium and uxdesign.cc were not reachable from the research environment; facts from them (Shopify Polaris, Spotify "5 things") are confirmed through search excerpts.

- Pinterest / Gestalt — figma.com/blog/how-pinterests-design-systems-team-measures-adoption/
- Uber / Base — uber.com/us/en/blog/design-system-at-scale/; luciapineda.me/base-adoption-dashboard
- IBM / Carbon for IBM.com — knapsack.cloud/blog/lessons-learned-from-working-on-carbon-for-ibm-com; robin-cannon.com/p/design-system-adoption-numbersjust
- Sparkbox, Carbon experiment — sparkbox.com/foundry/design_system_roi_impact_of_design_systems_business_value_carbon_design_system
- Sparkbox, maturity model and surveys — sparkbox.com/foundry/design_system_maturity_model; designsystemsurvey.sparkbox.com
- Spotify / Encore — iamtyce.com/blog/can-i-get-an-encore-spotifys-design-system-three-years-on; medium.com/spotify-design (excerpts)
- Shopify / Polaris — medium.com/shopify-ux/uplifting-shopify-polaris-7c54fc6564d9 (excerpts); Shopify DX survey — getdx.com/blog/shopify-developer-experience-survey/
- Mews — developers.mews.com/design-system-adoption-metric-building/
- Productboard — productboard.com/blog/how-we-measure-adoption-of-a-design-system-at-productboard/
- Brevo / Indigo — engineering.brevo.com/how-to-track-design-system-adoption/
- Optimizely / Axiom and Omlet customer practice — omlet.dev/blog/how-optimizely-uses-omlet-to-drive-efficient-design-system-adoption/; omlet.dev/blog/data-driven-design-systems-in-practice/; omlet.dev/blog/how-leaders-measure-design-system-adoption/
- Sprout Social / Seeds — chsmc.org/2020/08/design-system-health/
- Google / Material, Microsoft / Fluent — figma.com/blog/introducing-design-system-analytics/; designsystems.com/googles-material-design-a-design-system-with-mass-adoption/
- athenahealth, Squarespace, Swiggy — figma.com/blog/design-systems-104-making-metrics-matter/
- Figma, the 34% experiment — figma.com/blog/measuring-the-value-of-design-systems/
- Netflix / Hawkins — thedesignsystem.guide/interview-amber-feinerman
- Atlassian — uxpin.com/studio/blog/atlassian-design-system-creating-design-harmony-scale/
- Booking.com — ardakaracizmeli.com/article/design-system
- GitHub / Primer — github.com/primer/react, [contributor-docs/CONTRIBUTING.md](https://github.com/contributor-docs/CONTRIBUTING.md); IBM Carbon contributing — v10.carbondesignsystem.com/contributing/component/
- Nathan Curtis — nathanacurtis.substack.com/p/versioning-design-systems-48cceb5ace4d; eightshapes.com/articles/measuring-design-system-success/
- Dan Mall — v5.danmall.com/posts/in-search-of-a-better-design-system-metric-than-adoption/
- Knapsack — hubspot.knapsack.cloud/hubfs/Design-System-Insights-ROI-2021.pdf; knapsack.cloud/blog/why-design-system-adoption-isnt-the-true-measure-of-success
- Lucid — lucid.co/techblog/2023/01/09/learning-how-to-measure-a-design-systems-success
- Headway — headway.io/blog/design-system-adoption-vs-engagement
- DNSK "97% connected" — dnsk.work/blog/design-system-adoption-metrics-lie-measuring-installation-measuring-value/
- zeroheight — report.zeroheight.com (2026); Design Systems Report 2025; How We Document 2022–2024; zeroheight.com/blog/the-metrics-that-actually-matter-in-design-systems/
- Romina Kavcic — thedesignsystem.guide/design-system-metrics; Netguru — netguru.com/blog/design-system-metrics
- Atlassian Jira Control Chart — support.atlassian.com/jira-software-cloud/docs/view-and-understand-the-control-chart/